Лекция 1: От процедурного подхода к объектам. Инкапсуляция и защита инвариантов

Курс: Объектно-ориентированное программирование 1 (CS2203)
Университет: КазНУ им. аль-Фараби
Преподаватель: Санжар Аубакиров, PhD
Аудитория: Бакалавриат, 2-й курс, специальность «6B06110 Программная инженерия»
Формат: Лекция, 2 академических часа (90 минут)
Методологический нарратив: Chaos → Structure (От процедурного хаоса к объектной структуре)

Введение и контекст дисциплины

Добро пожаловать на курс «Объектно-ориентированное программирование 1».
На 1-м курсе в рамках курсов «Введение в программирование» и «Алгоритмы и структуры данных» вы научились писать работающий код, манипулировать массивами, реализовывать базовые структуры данных и оценивать их алгоритмическую сложность ($O(N)$, $O(N\log N)$).
Однако в индустриальной разработке написать код, который просто решает задачу здесь и сейчас — это меньше половины дела. Главный вызов промышленной программной инженерии — **управление сложностью растущей кодовой базы во времени**.
Когда проект разрастается от 50 строк до 500 000 строк, когда над системой работают десятки разработчиков, процедурный подход с глобальным состоянием и разрозненными функциями неизбежно погружает проект в состояние **Хаоса (Chaos)**. Изменение формата данных в одной части программы ломает десятки несвязанных модулей.
Цель нашего курса — освоить инженерную методологию перехода:

Chaos (Хаос процедурного спагетти) → Structure (Инкапсуляция и классы) → Order (SOLID и паттерны) → Emergence (Чистая архитектура)

В качестве сквозной прикладной предметной области мы будем использовать реальную распределенную систему городского транзита и процессинга платежей (**Avtobys / Smart City Transit**).

План лекции (90 минут)

№	Время	Тема	Формат
1	00:00–00:15	Проблема процедурного подхода: глобальное состояние, рассинхронизация и хрупкость	Анализ кейса
2	00:15–00:35	Объектная модель: идентичность, состояние,	Теория + код

№	Время	Тема	Формат
		поведение. Классы и экземпляры	
3	00:35–00:55	Инкапсуляция и сокрытие данных (Information Hiding). Защита инвариантов	Архитектурный разбор
4	00:55–01:15	Реализация в Python 3.12: свойства (@property), валидаторы, неизменяемость (frozen)	Практика
5	01:15–01:30	Типичные антипаттерны: Анемичная модель (Anemic Domain Model) vs Богатая модель	Инженерное обсуждение

---

1. Проблема процедурного подхода (Chaos)

Рассмотрим реальный пример: система оплаты проезда в общественном транспорте. Пассажир прикладывает карту или сканирует QR-код. В системе есть баланс карты, статус тарифа и история поездок.
В процедурном стиле данные хранятся в словарях или глобальных структурах, а функции оперируют ими извне:

```
# procedural_chaos.py
# Данные отделены от поведения и открыты всем на чтение и запись
passenger_card = {
    "card_id": "KZ-7701-9921",
    "balance": 250.0,
    "is_blocked": False,
    "discount_rate": 0.5 # студенческий тариф 50%
}

def pay_trip(card: dict, base_fare: float) -> bool:
    actual_fare = base_fare * card["discount_rate"]
    if card["balance"] >= actual_fare and not card["is_blocked"]:
        card["balance"] -= actual_fare
        return True
    return False
```

В чем фундаментальные уязвимости такого подхода?

1. **Отсутствие защиты инвариантов**: Любой другой модуль программы может напрямую выполнить:
   ```
   passenger_card["balance"] = -1000.0  # Отрицательный баланс!
   passenger_card["discount_rate"] = 1.5 # Скидка превратилась в наценку!
   ```

2. **Размытие бизнес-логики**: Если правила списания или валидации изменятся, программисту придется искать все места во всей кодовой базе, где читается или модифицируется словарь passenger_card.
3. **Отсутствие гарантированной инициализации**: Словарь может быть создан без обязательного поля "discount_rate", что приведет к KeyError в рантайме посреди ночи на валидаторе в автобусе.

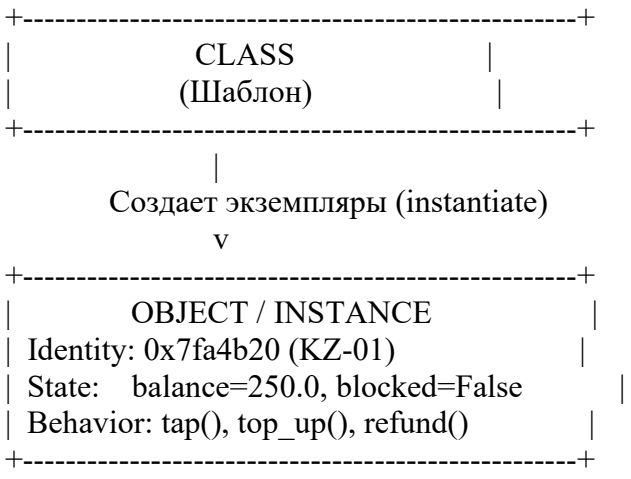
---

2. Понятие класса, объекта и идентичности

Объектно-ориентированное программирование связывает **состояние** и **поведение** в единую логическую сущность.

Три кита сущности в ООП:

1. **Идентичность (Identity)**: Уникальный адрес или идентификатор объекта в памяти, отличающий его от любых других объектов, даже если их внутренние поля полностью совпадают.
2. **Состояние (State)**: Совокупность значений внутренних атрибутов объекта в данный момент времени.
3. **Поведение (Behavior)**: Набор операций (методов), с помощью которых объект взаимодействует с внешним миром и изменяет свое состояние.



---

3. Инкапсуляция и сокрытие данных (Information Hiding)

Часто в поверхностных учебниках инкапсуляцию определяют просто как «добавление приватных полей и написание геттеров и сеттеров». Это грубейшее заблуждение.
Инкапсуляция — это объединение данных и методов работы с ними внутри компонента с одновременным **сокрытием внутреннего устройства** и предоставлением минимального, строго контролируемого публичного интерфейса.

Понятие инварианта

Инвариант — это логическое условие или предикат, который гарантированно должен оставаться истинным на протяжении всего жизненного цикла объекта (от завершения конструктора до уничтожения объекта).
Примеры бизнес-инвариантов транспортной карты: 1. Идентификатор карты card_id не может быть пустым и неизменен после создания. 2. Баланс карты balance никогда не может быть строго меньше нуля (balance ≥ 0). 3. Процент скидки discount_rate обязан

находиться в диапазоне [0.0; 1.0]. 4. Заблокированная карта (is_blocked == True) не может совершать транзакции списания.
Класс берет на себя **полную ответственность** за защиту этих инвариантов. Внешний клиент не имеет возможности нарушить инвариант, даже если попытается это сделать.

---

4. Проектирование Rich Domain Model на Python 3.12

Создадим правильную реализацию сущности транспортной карты.

```python
# domain/transit_card.py
from decimal import Decimal
from typing import Final


class InsufficientFundsError(Exception):
    """Исключение при нехватке средств на балансе карты."""
    pass


class CardBlockedError(Exception):
    """Исключение при попытке операции с заблокированной картой."""
    pass


class TransitCard:
    """
    Транспортная карта платежной системы Avtobys.
    Гарантирует защиту бизнес-инвариантов баланса и скидок.
    """

    MIN_TOP_UP_AMOUNT: Final[Decimal] = Decimal("100.00")

    def __init__(self, card_id: str, initial_balance: Decimal = Decimal("0.00"), discount_rate: Dec
imal = Decimal("0.00")):
        if not card_id or not card_id.strip():
            raise ValueError("Идентификатор карты (card_id) не может быть пустым.")

        if discount_rate < Decimal("0.00") or discount_rate > Decimal("1.00"):
            raise ValueError(f"Коэффициент скидки должен быть в пределах [0.0, 1.0], передано
: {discount_rate}")

        if initial_balance < Decimal("0.00"):
            raise ValueError(f"Начальный баланс не может быть отрицательным: {initial_balanc
e}")

        self._card_id: str = card_id.strip()
        self._balance: Decimal = initial_balance
        self._discount_rate: Decimal = discount_rate
        self._is_blocked: bool = False

    @property
    def card_id(self) -> str:
        """Публичный неизменяемый идентификатор карты."""
```

```
        return self._card_id

    @property
    def balance(self) -> Decimal:
        """Текущий баланс карты (доступен только для чтения снаружи)."""
        return self._balance

    @property
    def is_blocked(self) -> bool:
        return self._is_blocked

    def block(self, reason: str) -> None:
        """Блокировка карты при утере или мошенничестве."""
        self._is_blocked = True

    def unblock(self) -> None:
        """Разблокировка карты администратором."""
        self._is_blocked = False

    def top_up(self, amount: Decimal) -> None:
        """
        Пополнение баланса. Защищает инвариант минимальной суммы пополнения.
        """
        if self._is_blocked:
            raise CardBlockedError(f"Невозможно пополнить заблокированную карту {self._car
d_id}.")

        if amount < self.MIN_TOP_UP_AMOUNT:
            raise ValueError(f"Минимальная сумма пополнения: {self.MIN_TOP_UP_AMOUNT
} KZT, передано: {amount} KZT.")

        self._balance += amount

    def charge_fare(self, base_fare: Decimal) -> Decimal:
        """
        Списание за проезд с учетом персональной скидки.
        Возвращает фактически списанную сумму.
        """
        if self._is_blocked:
            raise CardBlockedError(f"Карта {self._card_id} заблокирована. Списание отклонено.
")

        if base_fare <= Decimal("0.00"):
            raise ValueError("Базовый тариф должен быть строго положительным.")

        effective_fare = base_fare * (Decimal("1.00") - self._discount_rate)

        if self._balance < effective_fare:
            raise InsufficientFundsError(
                f"Недостаточно средств на карте {self._card_id}. "
                f"Требуется: {effective_fare} KZT, доступно: {self._balance} KZT."
            )
```

```
        self._balance -= effective_fare
        return effective_fare

    def __repr__(self) -> str:
        status = "BLOCKED" if self._is_blocked else "ACTIVE"
        return f"<TransitCard id='{self._card_id}' balance={self._balance:.2f} KZT status={status
}>"
```

---

5. Анемичная модель (Антипаттерн) против Богатой модели

В промышленной разработке (особенно среди новичков) распространен антипаттерн **Anemic Domain Model** (Анемичная модель предметной области):

```
# АНТИПАТТЕРН: Анемичный класс
class BadTransitCard:
    def __init__(self):
        self.card_id = ""
        self.balance = 0.0
        self.is_blocked = False

# Логика размазана по внешним сервисам
class CardService:
    def deduct(self, card: BadTransitCard, amount: float):
        # Если разработчик забудет проверить блокировку здесь,
        # система позволит списать деньги с заблокированной карты!
        card.balance -= amount
```

Почему это опасно:

- Объект превращается в обычный пассивный мешок с данными (C-style struct).
- Инкапсуляция разрушена: ответственность за целостность данных лежит на сотнях внешних вызывающих функций.
- Поведение отделено от состояния, что сводит на нет все преимущества объектно-ориентированной парадигмы.

Принцип «Tell, Don’t Ask» (Приказывай, а не спрашивай)

Вместо того чтобы извлекать внутренности объекта, проверять их снаружи и перезаписывать, **попросите объект выполнить действие самостоятельно**: - ❌ Плохо: if card.balance >= fare and not card.is_blocked: card.balance -= fare - Отлично: actual_cost = card.charge_fare(fare)

Контрольные вопросы и выводы

1. Почему прямое изменение полей структуры данных приводит к хрупкости архитектуры в больших проектах?
2. Что такое инвариант класса и в какой момент жизненного цикла объекта он должен соблюдаться?
3. В чем разница между сокрытием данных (Data Hiding) и инкапсуляцией (Encapsulation)?
4. Каковы последствия применения антипаттерна «Анемичная доменная модель»?

5. Как принцип «Tell, Don’t Ask» повышает надежность объектной модели?

Литература к лекции

1. **Фаулер М.** Рефакторинг: улучшение проекта существующего кода. – М.: Диалектика, 2020. (Глава 3: Запахи кода — Data Clumps, Temporary Fields).
2. **Мартин Р.** Чистый код: создание, анализ и рефакторинг. – СПб.: Питер, 2019. (Глава 6: Объекты и структуры данных).
3. **Гамма Э. и др.** Приемы объектно-ориентированного проектирования. – СПб.: Питер, 2020.